JRodriguez_DSC680_P3M3Code

March 2, 2026

[45]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier,␣
↪HistGradientBoostingClassifier
from sklearn.metrics import roc_auc_score, precision_score, recall_score,␣
↪f1_score, confusion_matrix

pd.set_option("display.max_columns", None)

# Load dataset
inspections = pd.
↪read_csv("Environmental_Health_Restaurant_and_Market_Inspections.csv")

inspections.head()
```

```
/var/folders/7z/7bgphd555439sybhs6j0htjc0000gn/T/ipykernel_2115/743502371.py:13:
DtypeWarning: Columns (13) have mixed types. Specify dtype option on import or
set low_memory=False.
  inspections =
pd.read_csv("Environmental_Health_Restaurant_and_Market_Inspections.csv")
```

[45]:
```
   ACTIVITY DATE   OWNER ID             OWNER NAME FACILITY ID  \
0     01/01/2023  OW0100220  DREAMY CREATIONS, INC   FA0019651
1     01/01/2023  OW0257129        EVERYTABLE, PBC   FA0303137
2     01/01/2023  OW0296563     GG'S FOODHOUSE INC   FA0318639
3     01/01/2023  OW0298003  OMAR SANTOS HERNANDEZ   FA0320476
4     01/03/2023  OW0237369        H&R CORPORATION   FA0241484

       FACILITY NAME  RECORD ID            PROGRAM NAME PROGRAM STATUS  \
0  DREAMY CREATIONS  PR0008853        DREAMY CREATIONS       INACTIVE
1         EVERYTABLE  PR0260840     EVERYTABLE ACCOUNTS       INACTIVE
2    GG'S FOODHOUSE  PR0277939          GG'S FOODHOUSE         ACTIVE
3  PANADERIA LATINA  PR0280054  RESTAURANT WITH BAKERY       INACTIVE
4           7-ELEVEN  PR0189600                7-ELEVEN         ACTIVE
```

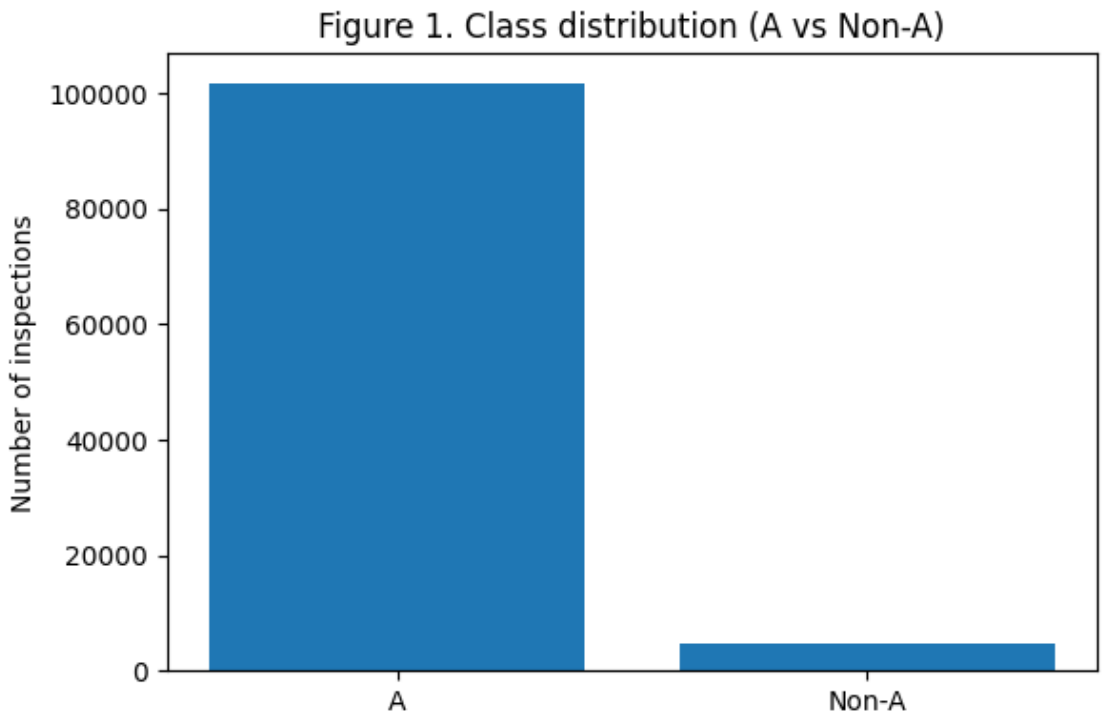
```
   PROGRAM ELEMENT                          PE DESCRIPTION  \
0             1631  RESTAURANT (0-30) SEATS MODERATE RISK
1             1630       RESTAURANT (0-30) SEATS LOW RISK
2             1631  RESTAURANT (0-30) SEATS MODERATE RISK
3             1632      RESTAURANT (0-30) SEATS HIGH RISK
4             1613   FOOD MKT RETAIL (2,000+ SF) LOW RISK

                    FACILITY ADDRESS FACILITY CITY FACILITY STATE  \
0  19441 BUSINESS CENTER DR STE #114    NORTHRIDGE             CA
1                     9002 TAMPA AVE    NORTHRIDGE             CA
2                  14419 SHERMAN WAY      VAN NUYS             CA
3                    7141 KESTER AVE      VAN NUYS             CA
4                 3477 W OLYMPIC BLVD   LOS ANGELES             CA

  FACILITY ZIP  SERVICE CODE SERVICE DESCRIPTION  SCORE GRADE SERIAL NUMBER  \
0        91324             1  ROUTINE INSPECTION    100     A     DA3992160
1        91324             1  ROUTINE INSPECTION    100     A     DA4065707
2        91405             1  ROUTINE INSPECTION    100     A     DA3855852
3        91495             1  ROUTINE INSPECTION    100     A     DA3943406
4        90019             1  ROUTINE INSPECTION     97     A     DAYG7QRMP

  EMPLOYEE ID
0   EE0000922
1   EE0000922
2   EE0000922
3   EE0000922
4   EE0001289
```

[46]:
```
# Cleaning the bare minimum fields so modeling does not get weird:
# date needs to be a real date, grade needs to be consistent, score should be␣
↪numeric.
# Then I am creating the target: 1 = non A, 0 = A.

# Parse dates
inspections["ACTIVITY DATE"] = pd.to_datetime(
    inspections["ACTIVITY DATE"].astype(str).str.strip(),
    errors="coerce"
)

# Normalize grade
inspections["GRADE"] = inspections["GRADE"].astype(str).str.strip().str.upper()

# Numeric score
inspections["SCORE"] = pd.to_numeric(inspections["SCORE"], errors="coerce")

# Keep only rows with the essentials
```

```
inspections = inspections.dropna(subset=["ACTIVITY DATE", "FACILITY ID",␣
↪"GRADE"]).copy()

# Keep A/B/C only (common for LA County exports)
inspections = inspections[inspections["GRADE"].isin(["A", "B", "C"])].copy()

# Target: Non-A = 1, A = 0
inspections["target_non_A"] = (inspections["GRADE"] != "A").astype(int)

inspections["target_non_A"].value_counts(dropna=False)

# Note: non A is only about 4 to 5 percent of the data. So if a model predicts␣
↪A every time, it will look "accurate" but be useless. That is why I am going␣
↪to focus on ROC AUC plus recall, not just accuracy.
```

[46]:
```
target_non_A
0    101833
1      4861
Name: count, dtype: int64
```

[71]:
```
# Class imbalance (A vs Non-A)
# This is just to make the imbalance obvious visually. It also sets␣
↪expectations for recall.

counts = inspections["target_non_A"].value_counts().sort_index()
labels = ["A", "Non-A"]

plt.figure(figsize=(6,4))
plt.bar(labels, counts.values)
plt.title("Figure 1. Class distribution (A vs Non-A)")
plt.ylabel("Number of inspections")
plt.tight_layout()
plt.show()

print("Non-A rate:", round(inspections["target_non_A"].mean(), 4))
```




```
Non-A rate: 0.0456
```

[47]:
```python
# Before I build any history features, I need the data in the right order.
# Everything that comes next depends on each facility being sorted by date.

inspections = inspections.sort_values(["FACILITY ID", "ACTIVITY DATE"]).copy()
```

[48]:
```python
# Creating facility level history features.
# The whole point is: when I am predicting an inspection, I only want to use␣
↪what was known BEFORE that inspection.
# shift(1) is what prevents leakage.

facility_group = inspections.groupby("FACILITY ID", sort=False)

# How many inspections happened BEFORE this row
inspections["prior_inspection_count"] = facility_group.cumcount()

# Prior average score (uses only previous scores)
inspections["prior_avg_score"] = (
    facility_group["SCORE"]
    .apply(lambda s: s.shift(1).expanding(min_periods=1).mean())
    .reset_index(level=0, drop=True)
)
```

```python
# Prior non-A rate (uses only previous grades)
inspections["prior_nonA_rate"] = (
    facility_group["target_non_A"]
    .apply(lambda s: s.shift(1).expanding(min_periods=1).mean())
    .reset_index(level=0, drop=True)
)

# Time since last inspection
inspections["prev_activity_date"] = facility_group["ACTIVITY DATE"].shift(1)
inspections["days_since_last_inspection"] = (
    inspections["ACTIVITY DATE"] - inspections["prev_activity_date"]
).dt.days

# Drop helper col
inspections.drop(columns=["prev_activity_date"], inplace=True)

inspections[[
    "FACILITY ID", "ACTIVITY DATE", "SCORE", "GRADE",
    "prior_inspection_count", "prior_avg_score", "prior_nonA_rate",␣
 ↪"days_since_last_inspection"
]].head(10)

# Small but important detail: the first inspection for each facility has no␣
 ↪history, so those rows will have nulls for the history features. That is␣
 ↪expected.
```

[48]:

```
       FACILITY ID ACTIVITY DATE  SCORE GRADE  prior_inspection_count  \
37075   FA0000999    2023-12-19     93     A                       0
59624   FA0000999    2024-07-02     98     A                       1
5304    FA0001114    2023-02-08     83     B                       0
72706   FA0001114    2024-11-13     90     A                       1
33849   FA0001155    2023-11-30     96     A                       0
8019    FA0001334    2023-02-28     93     A                       0
61457   FA0001334    2024-07-25     98     A                       1
45835   FA0001348    2024-03-01     99     A                       0
87035   FA0001348    2025-04-23     98     A                       1
14261   FA0001371    2023-04-29     94     A                       0

       prior_avg_score  prior_nonA_rate  days_since_last_inspection
37075              NaN              NaN                         NaN
59624             93.0              0.0                       196.0
5304               NaN              NaN                         NaN
72706             83.0              1.0                       644.0
33849              NaN              NaN                         NaN
8019               NaN              NaN                         NaN
61457             93.0              0.0                       513.0
45835              NaN              NaN                         NaN
```

```
87035              99.0              0.0                          418.0
14261               NaN              NaN                            NaN
```

[49]:
```
# For modeling, I am dropping first time inspections since there is no history␣
↪to learn from yet.
# This keeps the features consistent and avoids a bunch of missing values.

model_data = inspections[inspections["prior_inspection_count"] > 0].copy()

print("Rows used for modeling:", len(model_data))
print("Non-A rate:", model_data["target_non_A"].mean().round(4))
```

```
Rows used for modeling: 63782
Non-A rate: 0.0502
```

[50]:
```
# These are the features I actually want the model to learn from.
# I am mixing history (prior scores, prior non A rate, inspection frequency)␣
↪with context (risk, type, location).

feature_columns = [
    "prior_inspection_count",
    "prior_avg_score",
    "prior_nonA_rate",
    "days_since_last_inspection",
    "PROGRAM STATUS",
    "SERVICE DESCRIPTION",
    "PE DESCRIPTION",
    "FACILITY CITY",
    "FACILITY ZIP"
]

# Keeping only columns that actually exist
feature_columns = [c for c in feature_columns if c in model_data.columns]
print("Features used:", feature_columns)
```

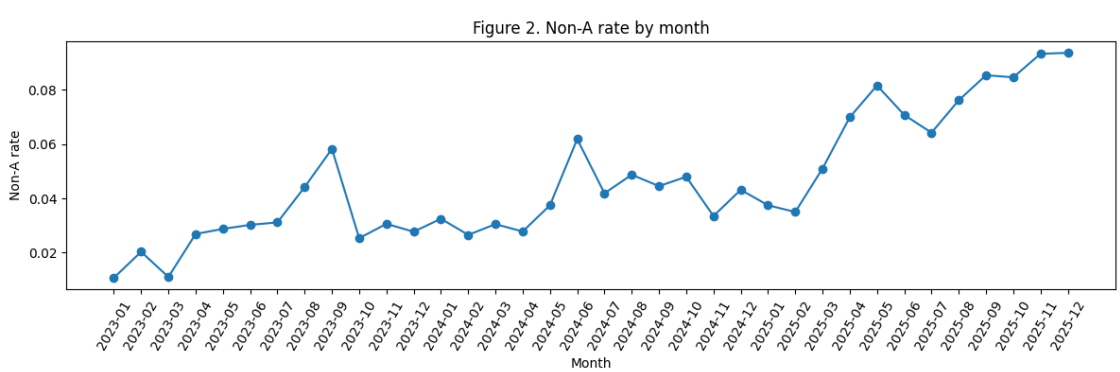
```
Features used: ['prior_inspection_count', 'prior_avg_score', 'prior_nonA_rate',
'days_since_last_inspection', 'PROGRAM STATUS', 'SERVICE DESCRIPTION', 'PE
DESCRIPTION', 'FACILITY CITY', 'FACILITY ZIP']
```

[51]:
```
# I am doing a time based split instead of random.
# Reason: I want this to behave like a real prediction problem.
# Train on earlier inspections, test on later inspections.

cutoff_date = model_data["ACTIVITY DATE"].quantile(0.80)

train_data = model_data[model_data["ACTIVITY DATE"] <= cutoff_date].copy()
test_data  = model_data[model_data["ACTIVITY DATE"] >  cutoff_date].copy()
```

```
print("Cutoff date:", cutoff_date.date())
print("Train rows:", len(train_data), "Test rows:", len(test_data))
print("Train non-A rate:", train_data["target_non_A"].mean().round(4))
print("Test non-A rate:", test_data["target_non_A"].mean().round(4))

# The non A rate is higher in the test period than the train period. That is a␣
↪good reminder that this data shifts over time, and it is also why I am not␣
↪doing a random split.
```

```
Cutoff date: 2025-06-17
Train rows: 51120 Test rows: 12662
Train non-A rate: 0.0428
Test non-A rate: 0.0804
```

[73]:
```
# Monthly Non-A rate over time
# Visual check for drift. If the non A rate changes over time, the model needs␣
↪to generalize across that.


model_data["month"] = model_data["ACTIVITY DATE"].dt.to_period("M").astype(str)

monthly_nonA = (
    model_data.groupby("month")["target_non_A"]
    .mean()
    .reset_index(name="nonA_rate")
)

plt.figure(figsize=(12,4))
plt.plot(monthly_nonA["month"], monthly_nonA["nonA_rate"], marker="o")
plt.xticks(rotation=60)
plt.title("Figure 2. Non-A rate by month")
plt.xlabel("Month")
plt.ylabel("Non-A rate")
plt.tight_layout()
plt.show()

monthly_nonA.tail()
```



[73]:

	month	nonA_rate
31	2025-08	0.076126
32	2025-09	0.085353
33	2025-10	0.084568
34	2025-11	0.093238
35	2025-12	0.093590

[52]:

```
# One-Hot Encode Categories
# Turning categorical columns into numeric columns the models can use.

X_train_raw = train_data[feature_columns].copy()
X_test_raw  = test_data[feature_columns].copy()

y_train = train_data["target_non_A"]
y_test  = test_data["target_non_A"]

# One-hot encode categorical columns
X_train = pd.get_dummies(X_train_raw, drop_first=True)
X_test  = pd.get_dummies(X_test_raw, drop_first=True)

# Align columns so train/test match
X_train, X_test = X_train.align(X_test, join="left", axis=1, fill_value=0)

X_train.shape, X_test.shape
```

[52]: ((51120, 827), (12662, 827))

[83]:

```
# Train and Compare Models + Metrics
# Trying a few models so I am not overcommitting to one approach.
# Logistic regression is the baseline.
# Random forest and gradient boosting handle nonlinear relationships better.

models = {
    "Logistic Regression": LogisticRegression(max_iter=5000,␣
 ↪class_weight="balanced", solver="liblinear"),
    "Random Forest": RandomForestClassifier(n_estimators=300, random_state=42,␣
 ↪class_weight="balanced_subsample", n_jobs=-1),
    "Gradient Boosting": HistGradientBoostingClassifier(random_state=42)
}

results = []
trained_models = {}

for model_name, model in models.items():
    model.fit(X_train, y_train)
```

```
    trained_models[model_name] = model

    y_pred = model.predict(X_test)

    # ROC AUC needs probabilities. HistGradientBoosting has predict_proba in␣
↪recent sklearn.
    if hasattr(model, "predict_proba"):
        y_prob = model.predict_proba(X_test)[:, 1]
        auc = roc_auc_score(y_test, y_prob)
    else:
        auc = np.nan

    results.append({
        "Model": model_name,
        "ROC-AUC": auc,
        "Precision": precision_score(y_test, y_pred, zero_division=0),
        "Recall": recall_score(y_test, y_pred, zero_division=0),
        "F1": f1_score(y_test, y_pred, zero_division=0)
    })

results_df = pd.DataFrame(results).sort_values("ROC-AUC", ascending=False)
results_df

# ROC AUC is useful for comparing ranking power, but it does not tell the whole␣
↪story. With rare outcomes like non A, I need to look at recall and the␣
↪confusion matrix, otherwise I can end up with a model that looks good but␣
↪catches almost nothing.
```

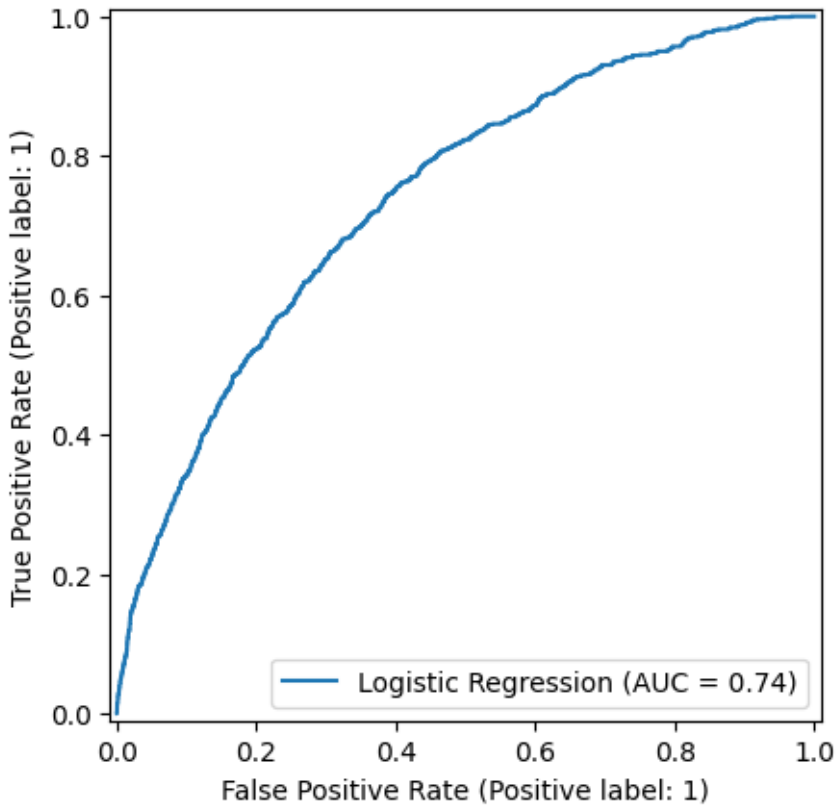
[83]:

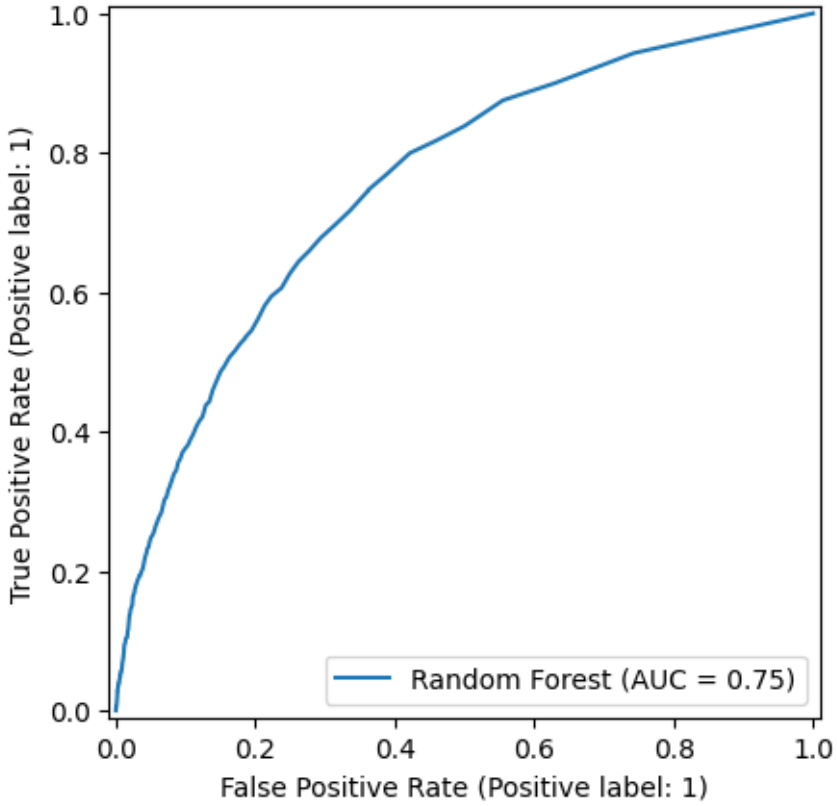
	Model	ROC-AUC	Precision	Recall	F1
2	Gradient Boosting	0.784592	0.491228	0.027505	0.052093
1	Random Forest	0.754975	0.475000	0.037328	0.069217
0	Logistic Regression	0.740687	0.182618	0.530452	0.271698

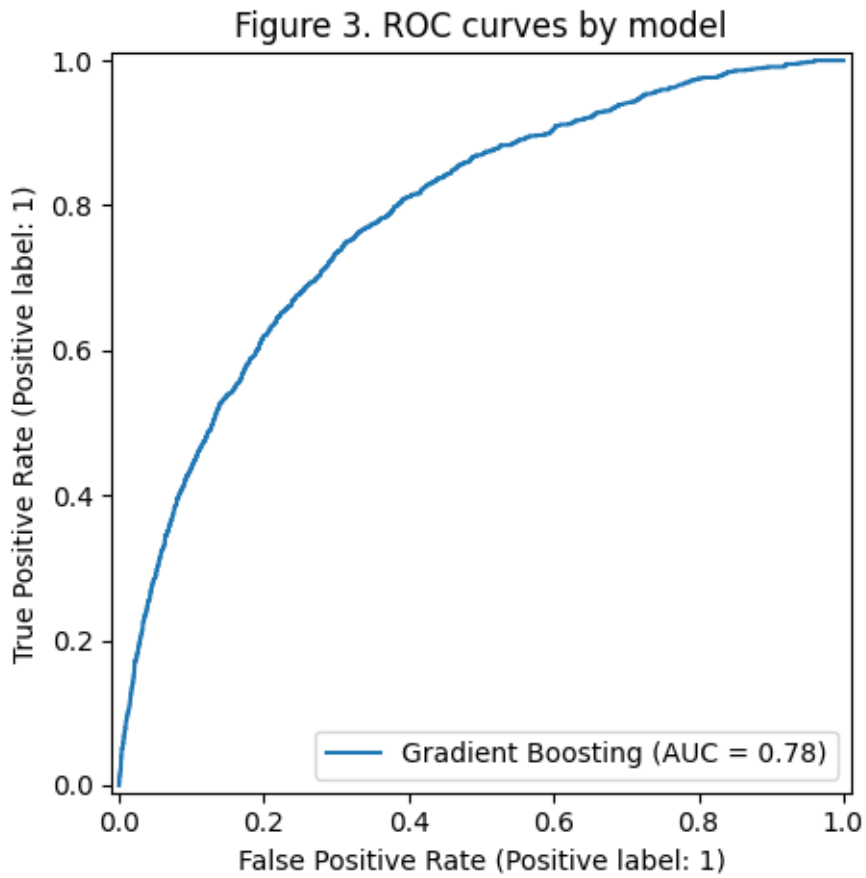
[85]:

```
from sklearn.metrics import RocCurveDisplay

# ROC curves are a clean way to compare models across thresholds.

plt.figure(figsize=(7,6))

for model_name, model in trained_models.items():
    if hasattr(model, "predict_proba"):
        y_prob = model.predict_proba(X_test)[:, 1]
        RocCurveDisplay.from_predictions(y_test, y_prob, name=model_name)

plt.title("Figure 3. ROC curves by model")
plt.tight_layout()
plt.show()
```

```
<Figure size 700x600 with 0 Axes>
```




1.0
0.8
0.6
0.4
0.2
0.0
True Positive Rate (Positive label: 1)
0.0
0.2
0.4
0.6
0.8
1.0
False Positive Rate (Positive label: 1)
Random Forest (AUC = 0.75)

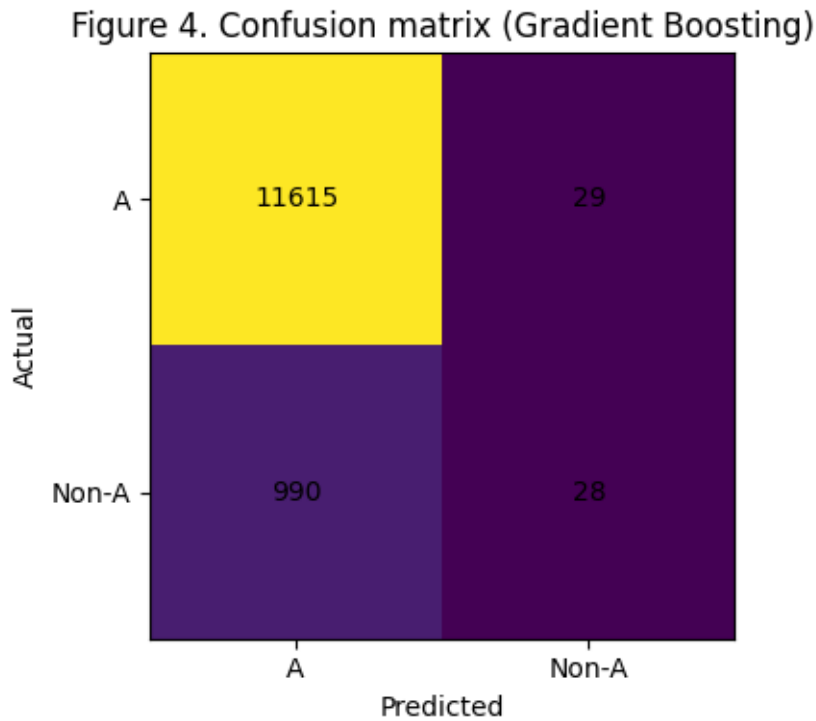


[54]:
```
# Confusion Matrix + Report for the Best Model
# Now I am picking the best model from the table and looking at what it␣
↪actually does.
# This is where the precision recall tradeoff becomes real.

from sklearn.metrics import classification_report

best_model_name = results_df.iloc[0]["Model"]
best_model = trained_models[best_model_name]

best_pred = best_model.predict(X_test)

print("Best model:", best_model_name)
print("\nClassification report:\n")
print(classification_report(y_test, best_pred, zero_division=0))

cm = confusion_matrix(y_test, best_pred)
cm
```

```
Best model: Gradient Boosting

Classification report:

              precision    recall  f1-score   support

           0       0.92      1.00      0.96     11644
           1       0.49      0.03      0.05      1018

    accuracy                           0.92     12662
   macro avg       0.71      0.51      0.51     12662
weighted avg       0.89      0.92      0.89     12662
```

[54]:
```
array([[11615,    29],
       [  990,    28]])
```

[77]:
```python
# Confusion matrix (best model at default threshold)

plt.figure(figsize=(5,4))
plt.imshow(cm)
plt.title(f"Figure 4. Confusion matrix ({best_model_name})")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.xticks([0,1], ["A", "Non-A"])
plt.yticks([0,1], ["A", "Non-A"])

for (i, j), v in np.ndenumerate(cm):
    plt.text(j, i, str(v), ha="center", va="center")

plt.tight_layout()
plt.show()

tn, fp, fn, tp = cm.ravel()
print("TP (caught non-A):", tp)
print("FN (missed non-A):", fn)
print("FP (false alarms):", fp)
```



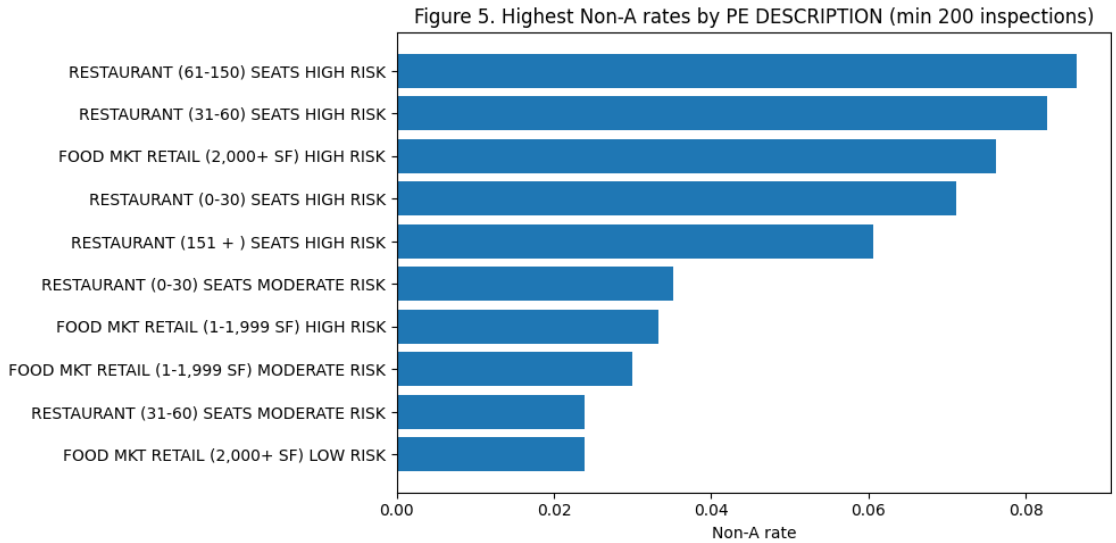

```
TP (caught non-A): 28
FN (missed non-A): 990
FP (false alarms): 29
```

[56]:
```python
# Simple Insights
# This part is less about modeling and more about pattern spotting.
# I want to see which inspection contexts and risk groupings have higher non A␣
↪rates.


# Non-A rate by service description (inspection context)
nonA_by_service = (
    model_data.groupby("SERVICE DESCRIPTION")["target_non_A"]
    .agg(nonA_rate="mean", inspections="count")
    .sort_values("nonA_rate", ascending=False)
)

# Non-A rate by risk description embedded in PE DESCRIPTION
nonA_by_pe = (
    model_data.groupby("PE DESCRIPTION")["target_non_A"]
    .agg(nonA_rate="mean", inspections="count")
    .sort_values("nonA_rate", ascending=False)
)
```

```
nonA_by_service.head(10), nonA_by_pe.head(10)
```

[56]:
```
(                            nonA_rate  inspections
 SERVICE DESCRIPTION
 ROUTINE INSPECTION           0.050343        61915
 OWNER INITIATED INSPECTION   0.047134         1867,
                                            nonA_rate  inspections
 PE DESCRIPTION
 FOOD PROCESSING WHOLESALE (2,000-5,999 SF)   1.000000            1
 RESTAURANT (61-150) SEATS HIGH RISK          0.086456         6778
 RESTAURANT (31-60) SEATS HIGH RISK           0.082648         8022
 FOOD MKT RETAIL (2,000+ SF) HIGH RISK        0.076119          670
 RESTAURANT (0-30) SEATS HIGH RISK            0.071074        12072
 RESTAURANT (151 + ) SEATS HIGH RISK          0.060606         3300
 RESTAURANT (0-30) SEATS MODERATE RISK        0.035228        11241
 FOOD MKT RETAIL (1-1,999 SF) HIGH RISK       0.033308         2642
 FOOD MKT RETAIL (1-1,999 SF) MODERATE RISK   0.029982          567
 RESTAURANT (31-60) SEATS MODERATE RISK       0.023860         1886)
```

[79]:
```
# Highest Non-A rates by PE DESCRIPTION (with volume filter)
# PE DESCRIPTION is basically the built in risk grouping in this dataset.
# I am filtering out tiny categories so I do not overreact to one off results.


nonA_by_pe_df = (
    model_data.groupby("PE DESCRIPTION")["target_non_A"]
    .agg(nonA_rate="mean", inspections="count")
    .reset_index()
)

# Filter out tiny groups so we don't overreact to 1 record = 100%
nonA_by_pe_df = nonA_by_pe_df[nonA_by_pe_df["inspections"] >= 200].copy()

top_pe = nonA_by_pe_df.sort_values("nonA_rate", ascending=False).head(10)

plt.figure(figsize=(10,5))
plt.barh(top_pe["PE DESCRIPTION"], top_pe["nonA_rate"])
plt.gca().invert_yaxis()
plt.title("Figure 5. Highest Non-A rates by PE DESCRIPTION (min 200␣
 ↪inspections)")
plt.xlabel("Non-A rate")
plt.tight_layout()
plt.show()

top_pe
```



[79]:

	PE DESCRIPTION	nonA_rate	inspections
22	RESTAURANT (61-150) SEATS HIGH RISK	0.086456	6778
19	RESTAURANT (31-60) SEATS HIGH RISK	0.082648	8022
5	FOOD MKT RETAIL (2,000+ SF) HIGH RISK	0.076119	670
13	RESTAURANT (0-30) SEATS HIGH RISK	0.071074	12072
16	RESTAURANT (151 +) SEATS HIGH RISK	0.060606	3300
15	RESTAURANT (0-30) SEATS MODERATE RISK	0.035228	11241
2	FOOD MKT RETAIL (1-1,999 SF) HIGH RISK	0.033308	2642
4	FOOD MKT RETAIL (1-1,999 SF) MODERATE RISK	0.029982	567
21	RESTAURANT (31-60) SEATS MODERATE RISK	0.023860	1886
6	FOOD MKT RETAIL (2,000+ SF) LOW RISK	0.023851	3480

[ ]: